

GÉNÉRATEUR DE MOT DE PASSE EN PYTHON

De zéro à un outil que tu vas vraiment utiliser



#vCODE2

Tu as créé ton premier outil sécurisé

TABLE DES MATIÈRES

1. Introduction - Ce qu'on va construire
2. Les bases Python nécessaires
3. Étape 1 : Afficher un mot de passe fixe
4. Étape 2 : Choisir un caractère au hasard
5. Étape 3 : Répéter pour avoir plusieurs caractères
6. Étape 4 : Assembler en mot de passe complet
7. Étape 5 : Personnaliser la longueur
8. Étape 6 : Améliorer la sécurité
9. Version finale complète
10. Aller plus loin

INTRODUCTION

CE QU'ON VA CONSTRUIRE ENSEMBLE

Le problème

Tu utilises le même mot de passe partout ? "azerty123" ou "motdepasse" ?

Mauvaise idée. Un seul site piraté = tous tes comptes compromis.

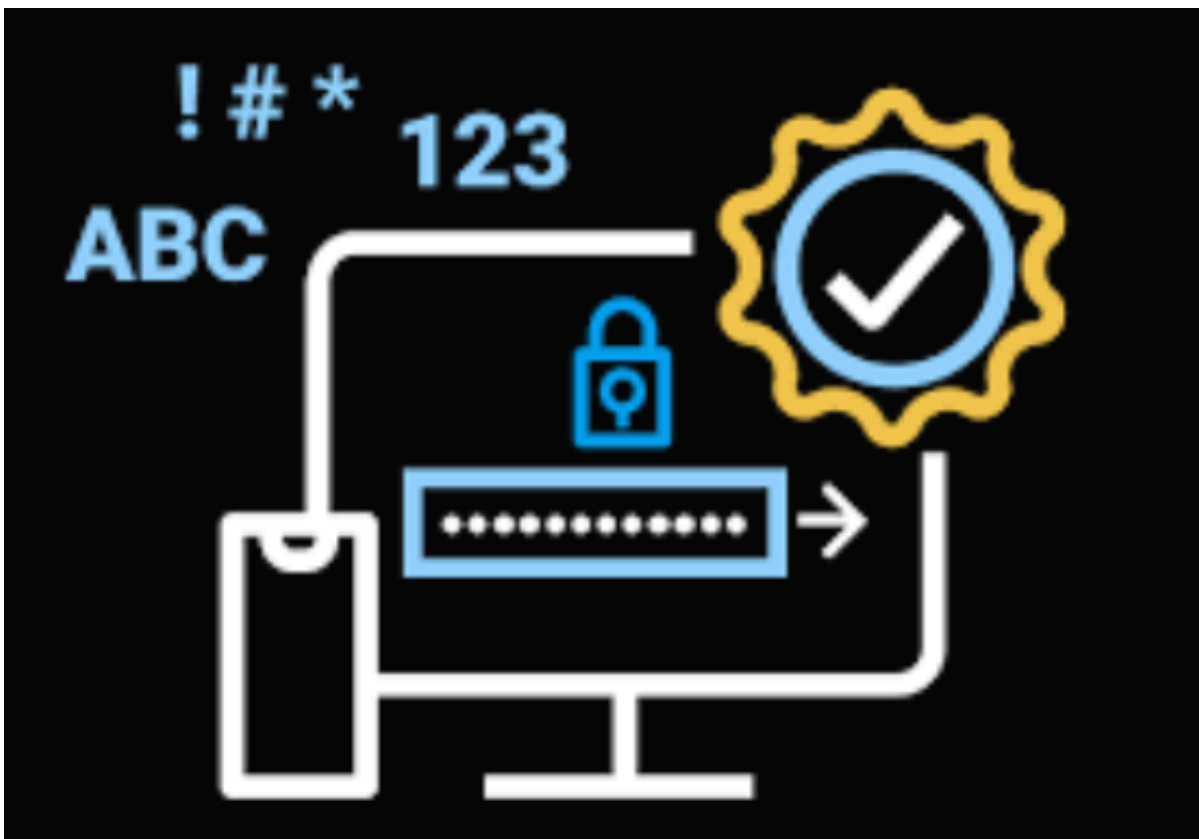
La solution

Un **générateur de mot de passe** qui crée des mots de passe aléatoires et sécurisés comme :

- A7k#mP2\$xQ9!
- zR5@nL8!pW3%
- B4&hT9#vK2\$m

Ce que tu vas apprendre

1. **Variables** : Stocker des informations
2. **Chaînes de caractères** : Manipuler du texte
3. **Nombres aléatoires** : Créer du hasard
4. **Boucles** : Répéter des actions
5. **Fonctions** : Organiser ton code



Comment le générateur crée un mot de passe sécurisé

À la fin de cette session, tu auras un outil fonctionnel que tu pourras utiliser tous les jours.

LES BASES PYTHON

CE QU'IL FAUT SAVOIR AVANT DE COMMENCER

1. Afficher du texte

```
print("Bonjour")
```

Ce qui se passe :

- `print` est une **fonction** = une action
- `"Bonjour"` est une **chaîne de caractères** = du texte
- Le texte s'affiche dans le terminal

Résultat :

```
Bonjour
```

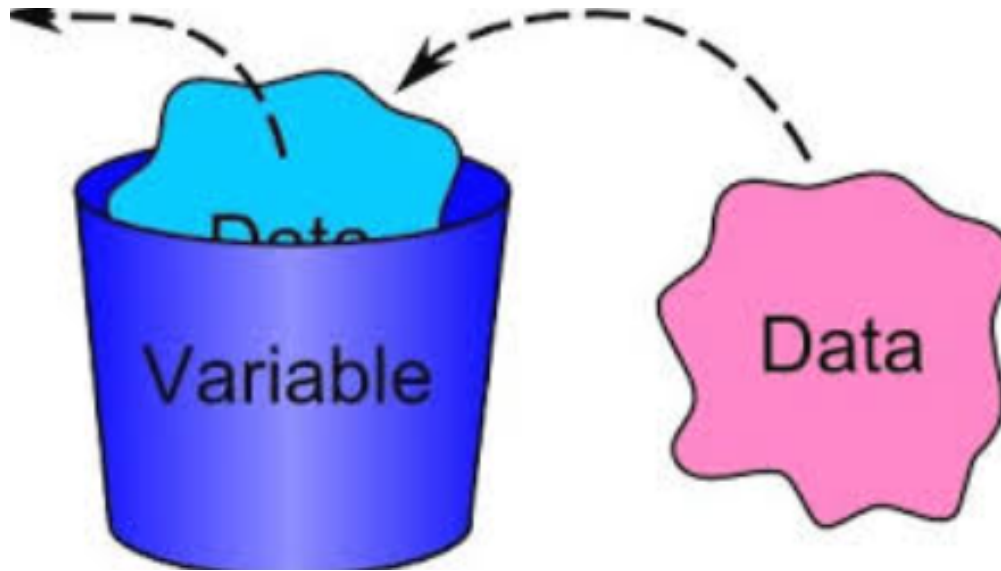
2. Les variables : stocker des informations

```
nom = "Pascal"  
print(nom)
```

Ce qui se passe :

- `nom` est une **variable** = une boîte avec un nom
- `=` signifie "ranger dans la boîte"
- `"Pascal"` est la valeur qu'on range
- `print(nom)` affiche le contenu de la boîte

Analogie : Une variable, c'est comme un tiroir étiqueté. Tu ranges quelque chose dedans et tu peux le ressortir plus tard.



Une variable stocke une valeur comme un tiroir étiqueté

Résultat :

Pascal

3. Les chaînes de caractères : manipuler du texte

```
lettre = "A"  
chiffre = "7"  
mot_de_passe = lettre + chiffre  
print(mot_de_passe)
```

Ce qui se passe :

- "A" est un **caractère** (une lettre)
- "7" est aussi un caractère (même si c'est un chiffre)
- + colle les caractères ensemble (**concaténation**)
- mot_de_passe contient maintenant "A7"

Résultat :

Important : "7" (avec guillemets) est différent de 7 (sans guillemets).

- "7" = texte
- 7 = nombre

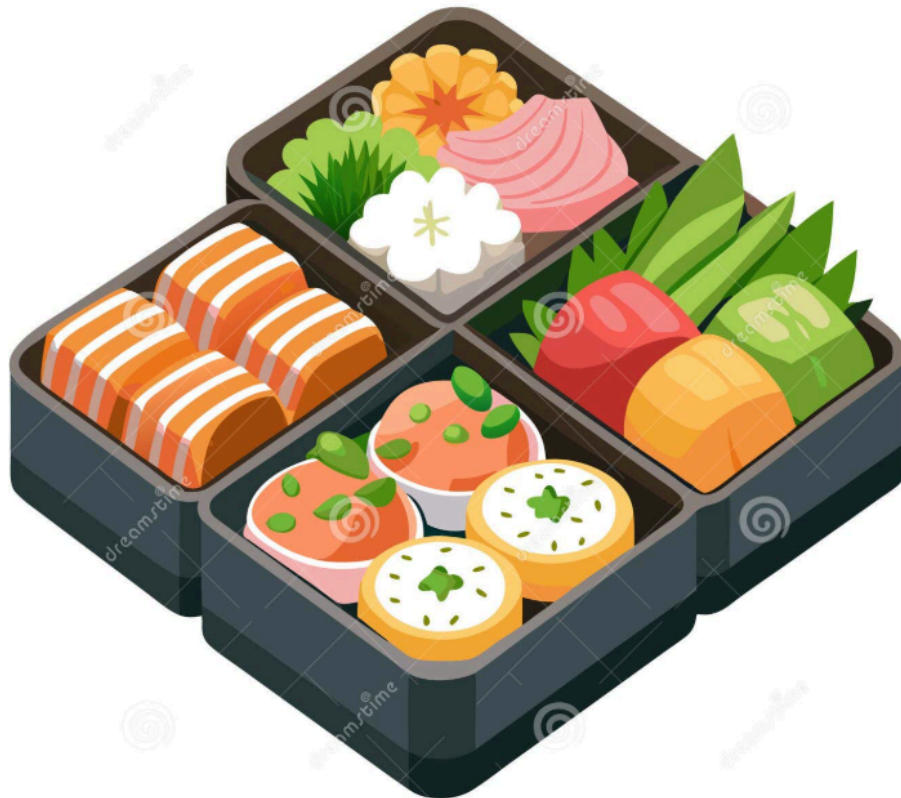
4. Les listes : plusieurs valeurs dans une variable

```
caracteres = ["A", "B", "C", "1", "2", "3"]  
print(caracteres)
```

Ce qui se passe :

- [] crée une **liste** = plusieurs valeurs dans une seule boîte
- On sépare les valeurs par des virgules
- caracteres contient maintenant 6 éléments

Analogie : Une liste, c'est comme un tiroir avec plusieurs compartiments.



Résultat :

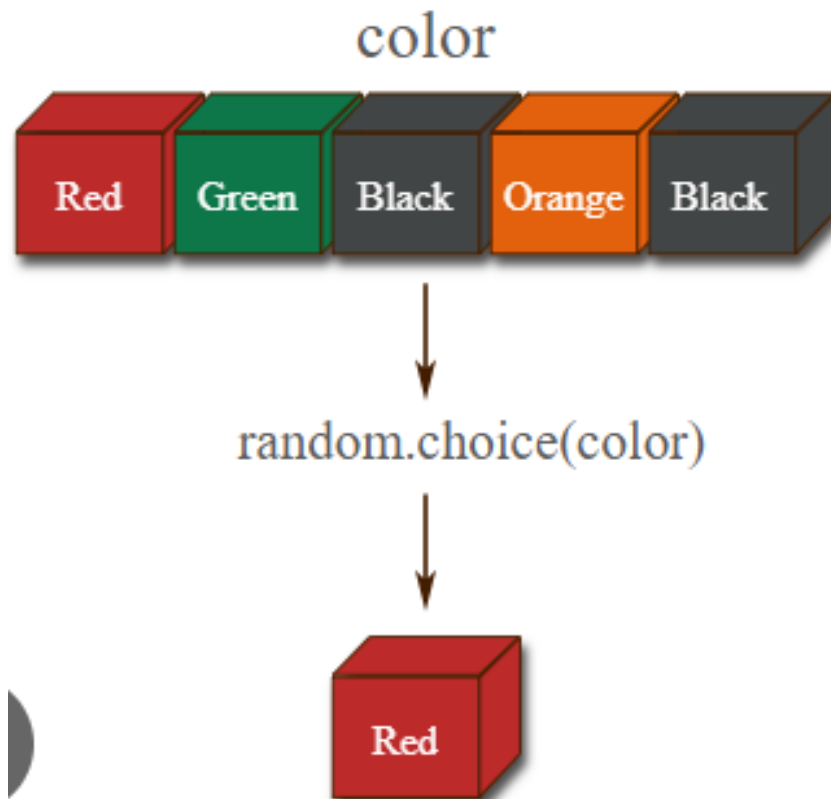
```
['A', 'B', 'C', '1', '2', '3']
```

5. Choisir un élément au hasard

```
import random

caracteres = ["A", "B", "C"]
choix = random.choice(caracteres)
print(choix)
```

Ce qui se passe :



random.choice() sélectionne un élément au hasard

..

- `import random` charge un **module** = une boîte à outils
- `random.choice()` est une **fonction** qui choisit au hasard
- `caracteres` est la liste dans laquelle on choisit

- `choix` contient le caractère choisi (A, B, ou C)

Résultat (exemple) :

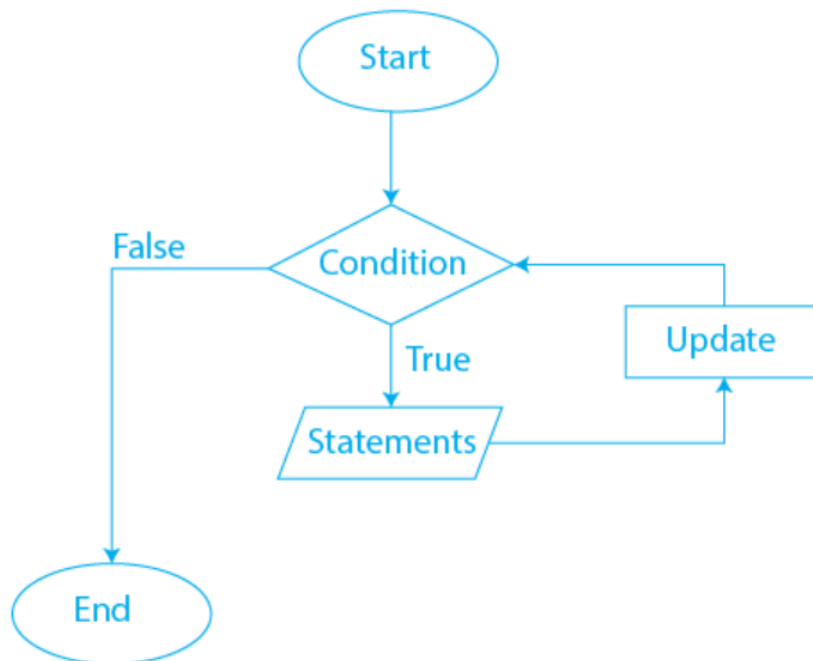
B

Important : Le résultat change à chaque fois que tu exécutes le code. C'est ça, l'aléatoire.

6. Répéter une action : les boucles

```
for i in range(5):  
    print("Bonjour")
```

Ce qui se passe :



La boucle répète l'action 8 fois pour 8 caractères

- `for` signifie "pour chaque"
- `i` est un **compteur** (on pourrait l'appeler autrement)
- `range(5)` signifie "de 0 à 4" (5 fois au total)
- `print("Bonjour")` s'exécute 5 fois

Résultat :

Bonjour
Bonjour
Bonjour
Bonjour
Bonjour

Analogie : C'est comme dire "répète cette action X fois".

ÉTAPE 1 : AFFICHER UN MOT DE PASSE FIXE

PREMIER PAS : UN MOT DE PASSE SIMPLE

Le code

```
mot_de_passe = "A7k#"
print("Votre mot de passe :", mot_de_passe)
```

Explication

1. On crée une **variable** `mot_de_passe`
2. On y range le texte `"A7k#"`
3. On affiche le texte "Votre mot de passe :" suivi du contenu de la variable

Résultat

Votre mot de passe : A7k#

Le problème

Ce mot de passe est toujours le même. Pas vraiment sécurisé.

Solution : On va le générer aléatoirement.

ÉTAPE 2 : CHOISIR UN CARACTÈRE AU HASARD

INTRODUCTION DU HASARD

Le code

```
import random

caracteres_possibles = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$$%"

caractere_choisi = random.choice(caracteres_possibles)
print("Caractère aléatoire :", caractere_choisi)
```

Explication ligne par ligne

Ligne 1 :

```
import random
```

→ On charge le module `random` qui contient des fonctions pour créer du hasard.

Ligne 3 :

```
caracteres_possibles = "ABC...xyz...123...!@#"
```

→ On crée une longue chaîne contenant TOUS les caractères qu'on peut utiliser :

- Majuscules : A à Z
- Minuscules : a à z
- Chiffres : 0 à 9
- Symboles : !@#\$%

Ligne 5 :

```
caractere_choisi = random.choice(caracteres_possibles)
```

→ `random.choice()` choisit UN caractère au hasard dans `caracteres_possibles`.

Ligne 6 :

```
print("Caractère aléatoire :", caractere_choisi)
```

→ On affiche le caractère choisi.

Résultat (exemple)

Caractère aléatoire : k

À chaque exécution, le caractère change.

Ce qu'on comprend

Python peut choisir aléatoirement dans une liste ou une chaîne de caractères.

C'est la base de notre générateur.

ÉTAPE 3 : RÉPÉTER POUR AVOIR PLUSIEURS CARACTÈRES

CRÉER UN MOT DE PASSE DE PLUSIEURS CARACTÈRES

Le code

```
import random

caracteres_possibles = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$$%"

for i in range(8):
    caractere = random.choice(caracteres_possibles)
    print(caractere)
```

Explication

Ligne 5 :

```
for i in range(8):
```

→ Répète l'action 8 fois (pour un mot de passe de 8 caractères).

Ligne 6 :

```
caractere = random.choice(caracteres_possibles)
```

→ À chaque tour de boucle, choisit un caractère au hasard.

Ligne 7 :

```
print(caractere)
```

→ Affiche le caractère choisi.

Résultat (exemple)

A
7
k

m
P
2
\$

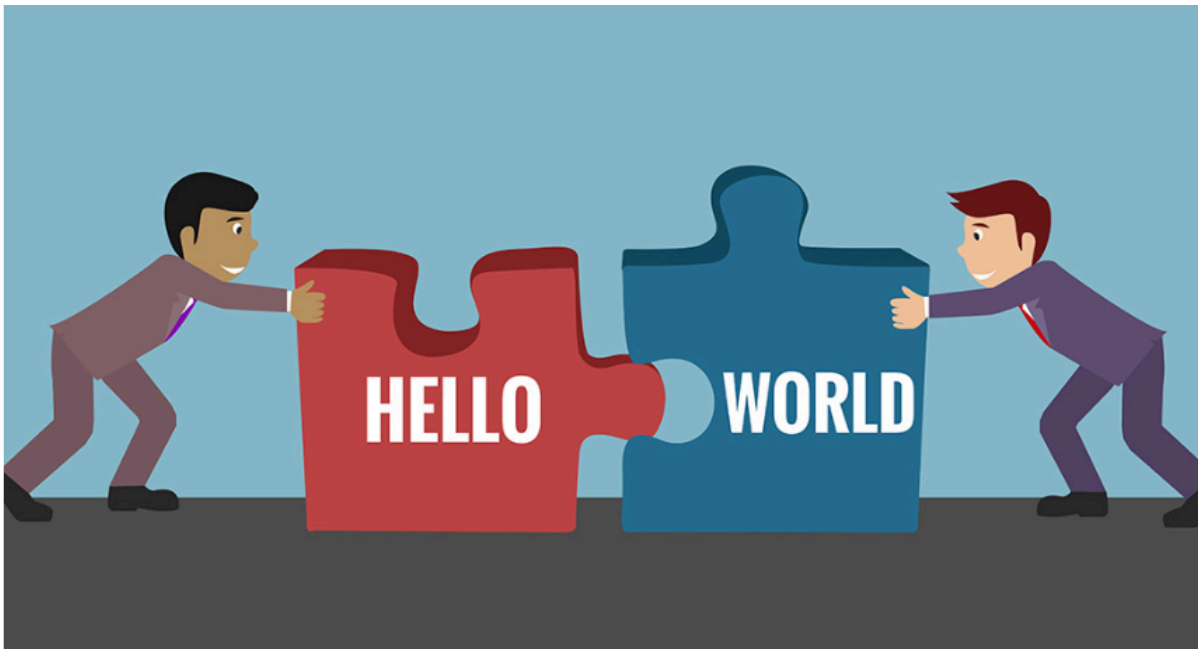
Le problème

Les caractères s'affichent ligne par ligne, pas assemblés en un seul mot de passe.

Solution : On va les coller ensemble.

ÉTAPE 4 : ASSEMBLER EN MOT DE PASSE COMPLET

CONSTRUIRE LE MOT DE PASSE FINAL



Le mot de passe se construit caractère par caractère

Le code

```
import random

caracteres_possibles = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#%$"

mot_de_passe = ""

for i in range(8):
    caractere = random.choice(caracteres_possibles)
    mot_de_passe = mot_de_passe + caractere

print("Votre mot de passe :", mot_de_passe)
```

Explication des nouveautés

Ligne 5 :

```
mot_de_passe = ""
```

→ On crée une variable vide (pour l'instant, elle ne contient rien).
C'est notre "boîte" où on va ranger les caractères au fur et à mesure.

Ligne 9 :

`mot_de_passe = mot_de_passe + caractere`

→ **C'est LA ligne magique.**

On prend le contenu actuel de `mot_de_passe` , on y ajoute le nouveau `caractere` , et on range le tout dans `mot_de_passe` .

Exemple d'évolution :

- Tour 1 : `mot_de_passe = "" + "A" → mot_de_passe = "A"`
- Tour 2 : `mot_de_passe = "A" + "7" → mot_de_passe = "A7"`
- Tour 3 : `mot_de_passe = "A7" + "k" → mot_de_passe = "A7k"`
- ... et ainsi de suite jusqu'à 8 caractères

Résultat (exemple)

Votre mot de passe : A7k#mP2\$

Ce qu'on comprend

En commençant avec une variable vide et en y ajoutant des caractères un par un, on construit progressivement le mot de passe complet.

C'est exactement comme construire un mot lettre par lettre.

ÉTAPE 5 : PERSONNALISER LA LONGUEUR

LAISSER L'UTILISATEUR CHOISIR

Le code

```
import random

caracteres_possibles = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$$%"

longueur = int(input("Quelle longueur pour votre mot de passe ? "))

mot_de_passe = ""

for i in range(longueur):
    caractere = random.choice(caracteres_possibles)
    mot_de_passe = mot_de_passe + caractere

print("Votre mot de passe :", mot_de_passe)
```

Explication des nouveautés

Ligne 5 :

```
longueur = int(input("Quelle longueur... ? "))
```

Décortiquons de l'intérieur vers l'extérieur :

1. `input("...")` affiche la question et attend que l'utilisateur tape quelque chose
2. Ce que l'utilisateur tape est du **texte** (même si c'est "12")
3. `int()` transforme le texte en **nombre entier**
4. `longueur` contient maintenant ce nombre

Pourquoi `int()` ?

Sans `int()` , si l'utilisateur tape "12", Python voit le texte "12", pas le nombre 12.

Or, `range()` a besoin d'un nombre, pas d'un texte.

Ligne 9 :

```
for i in range(longueur):
```

→ Au lieu de `range(8)` fixe, on utilise `range(longueur)` qui s'adapte.

Résultat (exemple)

Quelle longueur pour votre mot de passe ? 12

Votre mot de passe : A7k#mP2\$xQ9!

Ce qu'on comprend

`input()` permet de poser une question à l'utilisateur.

`int()` transforme la réponse en nombre utilisable dans le code.

Notre générateur devient interactif.

ÉTAPE 6 : AMÉLIORER LA SÉCURITÉ

GARANTIR UN MOT DE PASSE FORT

Le problème

Avec notre code actuel, il est **possible** (même si rare) que le mot de passe généré ne contienne que des lettres, ou que des chiffres.

Un bon mot de passe devrait **toujours** contenir :

- Au moins une majuscule
- Au moins une minuscule
- Au moins un chiffre
- Au moins un symbole

La solution

On va séparer les caractères en **catégories** et s'assurer qu'on prend au moins un caractère de chaque catégorie.

Le code amélioré

```
import random

majuscules = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
minuscules = "abcdefghijklmnopqrstuvwxyz"
chiffres = "0123456789"
symboles = "!@#$%^&*"

longueur = int(input("Quelle longueur pour votre mot de passe ? "))

mot_de_passe = ""

# On prend au moins un caractère de chaque catégorie
mot_de_passe = mot_de_passe + random.choice(majuscules)
mot_de_passe = mot_de_passe + random.choice(minuscules)
mot_de_passe = mot_de_passe + random.choice(chiffres)
mot_de_passe = mot_de_passe + random.choice(symboles)

# On complète avec des caractères aléatoires
tous_caracteres = majuscules + minuscules + chiffres + symboles

for i in range(longueur - 4):
    mot_de_passe = mot_de_passe + random.choice(tous_caracteres)

print("Votre mot de passe :", mot_de_passe)
```

Explication ligne par ligne

Lignes 3-6 :

```
majuscules = "ABC...XYZ"
minuscules = "abc...xyz"
chiffres = "0123...789"
symboles = "!@#$%^&*"

```

→ On sépare les caractères en 4 catégories distinctes.

Lignes 13-16 :


```
mot_de_passe = mot_de_passe + random.choice(majuscules)
mot_de_passe = mot_de_passe + random.choice(minuscules)
mot_de_passe = mot_de_passe + random.choice(chiffres)
mot_de_passe = mot_de_passe + random.choice(symboles)
```

→ On garantit qu'on prend **au moins un caractère** de chaque catégorie.
Après ces 4 lignes, `mot_de_passe` contient 4 caractères (un de chaque type).

Ligne 19 :

```
tous_caracteres = majuscules + minuscules + chiffres + symboles
```

→ On regroupe TOUS les caractères possibles dans une seule chaîne.

Ligne 21 :

```
for i in range(longueur - 4):
```

→ **Important :** On a déjà 4 caractères. Donc si l'utilisateur veut 12 caractères, on doit en ajouter 12 - 4 = 8 supplémentaires.

Lignes 22 :

```
mot_de_passe = mot_de_passe + random.choice(tous_caracteres)
```

→ On complète avec des caractères aléatoires (n'importe quelle catégorie).

Résultat (exemple)

```
Quelle longueur pour votre mot de passe ? 12
Votre mot de passe : Ap7!kMz$2Qx%
```

Ce qu'on comprend

En forçant au moins un caractère de chaque catégorie, on garantit que le mot de passe est toujours fort.

Le générateur est maintenant vraiment sécurisé.

VERSION FINALE COMPLÈTE

LE CODE FINAL AVEC TOUTES LES AMÉLIORATIONS

Le code complet

```
import random

def generer_mot_de_passe(longueur):
    """Génère un mot de passe sécurisé de la longueur demandée"""

    # Catégories de caractères
    majuscules = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    minuscules = "abcdefghijklmnopqrstuvwxyz"
    chiffres = "0123456789"
    symboles = "!@#$%^&*"

    # Vérification de la longueur minimale
    if longueur < 8:
        print("Attention : un mot de passe sécurisé doit contenir au moins 8 caractères")
        longueur = 8

    # Construction du mot de passe
    mot_de_passe = ""

    # Au moins un caractère de chaque catégorie
    mot_de_passe += random.choice(majuscules)
    mot_de_passe += random.choice(minuscules)
    mot_de_passe += random.choice(chiffres)
    mot_de_passe += random.choice(symboles)

    # Compléter avec des caractères aléatoires
    tous_caracteres = majuscules + minuscules + chiffres + symboles
    for i in range(longueur - 4):
        mot_de_passe += random.choice(tous_caracteres)

    # Mélanger les caractères pour plus de sécurité
    liste_caracteres = list(mot_de_passe)
    random.shuffle(liste_caracteres)
    mot_de_passe = ''.join(liste_caracteres)

    return mot_de_passe

# Programme principal
print("* GÉNÉRATEUR DE MOT DE PASSE SÉCURISÉ *\n")

longueur = int(input("Quelle longueur souhaitez-vous ? "))
```

```
mot_de_passe = generer_mot_de_passe(longueur)

print("\n✓ Votre mot de passe sécurisé :")
print(f" {mot_de_passe}\n")
print("Copiez-le et gardez-le en lieu sûr !")
```

Les nouveautés expliquées

Fonction `generer_mot_de_passe()` :

```
def generer_mot_de_passe(longueur):
```

→ On crée une **fonction** = un bloc de code réutilisable.

Au lieu de tout écrire en vrac, on organise le code dans une fonction qu'on peut appeler quand on veut.

Vérification de longueur :

```
if longueur < 8:
    print("Attention...")
    longueur = 8
```

→ Si l'utilisateur demande moins de 8 caractères, on force à 8 minimum (sécurité).

Opérateur += :

```
mot_de_passe += random.choice(majuscules)
```

→ C'est un raccourci pour `mot_de_passe = mot_de_passe + ...`

Ça fait exactement la même chose, mais c'est plus court à écrire.

Mélanger les caractères :

```
liste_caracteres = list(mot_de_passe)
random.shuffle(liste_caracteres)
mot_de_passe = ''.join(liste_caracteres)
```

Pourquoi mélanger ?

Sans mélange, le mot de passe commence **toujours** par : Majuscule + Minuscule + Chiffre +

Symbole.

Exemple : Ap7!... (prévisible)

Après mélange, les caractères sont dans un ordre complètement aléatoire.

Exemple : 7!pA... (imprévisible)

Comment ça marche ?

1. `list(mot_de_passe)` transforme le texte en liste de caractères
2. `random.shuffle()` mélange la liste au hasard
3. `''.join()` rassemble les caractères en un seul texte

F-string pour affichage :

```
print(f" {mot_de_passe}\n")
```

→ Le `f` avant les guillemets permet d'insérer des variables directement dans le texte.
`{mot_de_passe}` sera remplacé par la valeur de la variable.

Résultat final

* GÉNÉRATEUR DE MOT DE PASSE SÉCURISÉ *

Quelle longueur souhaitez-vous ? 16

✓ Votre mot de passe sécurisé :

7k!Ap\$2MzQx%9Rt@

Copiez-le et gardez-le en lieu sûr !

ALLER PLUS LOIN

IDÉES D'AMÉLIORATION

1. Générer plusieurs mots de passe

```
nombre = int(input("Combien de mots de passe voulez-vous ? "))
for i in range(nombre):
    mdp = generer_mot_de_passe(12)
    print(f"{i+1}. {mdp}")
```

Résultat :

1. 7k!Ap\$2MzQx%
2. Rt@9Bv#5Nx&W
3. Lm*8Dq!3Hy^Z

2. Sauvegarder dans un fichier

```
with open("mots_de_passe.txt", "w") as fichier:
    for i in range(5):
        mdp = generer_mot_de_passe(12)
        fichier.write(f"{mdp}\n")

print("✓ 5 mots de passe sauvegardés dans mots_de_passe.txt")
```

3. Copier automatiquement dans le presse-papiers

```
import pyperclip

mot_de_passe = generer_mot_de_passe(12)
pyperclip.copy(mot_de_passe)
print("✓ Mot de passe copié ! Faites Ctrl+V pour le coller")
```

Note : Nécessite d'installer le module `pyperclip` :

```
pip install pyperclip
```

4. Interface graphique simple

```
import tkinter as tk

def generer():
    mdp = generer_mot_de_passe(12)
    resultat.set(mdp)

fenetre = tk.Tk()
fenetre.title("Générateur de Mot de Passe")

bouton = tk.Button(fenetre, text="Générer", command=generer)
bouton.pack()

resultat = tk.StringVar()
label = tk.Label(fenetre, textvariable=resultat, font=("Courier", 14))
label.pack()

fenetre.mainloop()
```

RÉCAPITULATIF : CE QUE TU AS APPRIS

LES COMPÉTENCES ACQUISES

Notions Python maîtrisées

Notion	Description	Utilité
Variables	Stocker des valeurs	Base de toute programmation
Chaînes de caractères	Manipuler du texte	Travailler avec des mots et phrases
Listes	Grouper plusieurs valeurs	Organiser des données
Boucles	Répéter des actions	Automatiser les tâches répétitives
Conditions	Prendre des décisions	Adapter le comportement du code
Fonctions	Organiser le code	Réutiliser et structurer
Modules	Utiliser du code existant	Étendre les capacités de Python

Notion	Description	Utilité
Input	Interagir avec l'utilisateur	Créer des programmes interactifs
Hasard	Générer de l'aléatoire	Créer de l'imprévisibilité

Ce que tu peux maintenant créer

Avec ces bases, tu es capable de coder :

- ✓ **Jeu de dés** : `random.randint(1, 6)`
- ✓ **Calculatrice simple** : `+`, `-`, `*`, `/`
- ✓ **Convertisseur de température** : Celsius ↔ Fahrenheit
- ✓ **Générateur de nombres de loterie** : 6 nombres entre 1 et 49
- ✓ **Quiz interactif** : Questions avec réponses
- ✓ **Liste de courses** : Ajouter, afficher, supprimer

Tu as les bases pour créer des vrais programmes utiles.

CONCLUSION

LE CHEMIN PARCOURU

D'où tu es parti

Au début de cette session, tu ne connaissais peut-être pas Python.

Ou tu avais vu quelques bases sans vraiment comprendre à quoi ça servait.

Où tu es maintenant

Tu as créé un **outil fonctionnel** de A à Z.

Pas un exercice théorique. Un vrai programme que tu peux utiliser tous les jours.

Tu as compris :

- Comment Python stocke des informations
- Comment créer du hasard

- Comment répéter des actions
- Comment organiser du code
- Comment interagir avec l'utilisateur

Le principe de Code Premier Pas

"On n'apprend pas en copiant-collant du code trouvé sur Internet."

"On apprend en comprenant chaque ligne, chaque mot, chaque symbole."

"On apprend en construisant quelque chose d'utile, pas un exemple théorique."

La suite

Ce générateur de mot de passe, c'est ta première pierre.

Maintenant, tu peux construire le mur.

Prochain vCODE : On verra comment améliorer ce code, ou on attaquera un nouveau projet concret.

Rappelle-toi toujours :

Les bases solides d'aujourd'hui sont les succès de demain

Chaque ligne de code que tu comprends vraiment te rapproche de tes objectifs.



#vCODE2

 **Contact :** [NOUS CONTACTER](#)

 **Rejoignez notre communauté**



[LinkedIn](#)

[WhatsApp](#)

[Discord](#)

